

Provenance-Chained Résumé Optimisation

A complete account of what was built, how it works, why each decision was taken, what was measured, and where it fails. Every number here traces to `eval/RESULTS.md`, `docs/paper/paper.tex`, or a live run against the local stack.

PRIMARY BACKEND `gemini-3.1-flash-lite` · CROSS-CHECK `qwen2.5:14b`, `gemma3:4b` (Ollama)
EMBEDDINGS `all-mpnet-base-v2` · INGESTION `Docling` · ORCHESTRATION `LangGraph` · CALIBRATOR `Ridge`
SERVING `FastAPI` + `Next.js`, `Docker Compose` · ATS GROUND TRUTH `ats-screener @ 4105f77`

1 · The one-minute answer

if you are asked only one question, answer with this

What is it? A system that rewrites a résumé toward a job description and reports a match score — under a hard constraint that *every factual claim in the output must trace to a span in the original document*. An LLM proposes the rewrite; a deterministic, LLM-free gate verifies it and deletes anything unsupported.

What is the contribution? Not the rewriter — anyone can prompt an LLM to improve a résumé. The contribution is the *gate* and the measurement around it. We show that prompt grounding alone is insufficient (40 unsourced claims shipped across 30 pairs despite an explicit "never invent" instruction), and that a provenance-resolved gate reduces that to zero on two different model backends.

Why does it matter? Optimising a résumé for match score *actively rewards fabrication*: an invented skill is precisely the keyword the job asked for, so it raises every naïve metric. Truthfulness cannot be a by-product of a good prompt; it has to be a checkable property enforced outside the model.

40 unsourced claims shipped with prompt grounding alone (gate OFF, 30 pairs)	0 shipped with the gate ON — on both backends	0.874 provenance-attachment accuracy, independent human-annotated set	0.333 Spearman ρ vs real ATS engine (cosine baseline 0.229)
--	---	---	--

THE TRAP QUESTION — REHEARSE THIS ANSWER

“Gate-ON = 0 is impressive.” It is **not an empirical finding**. The gate deletes unsourced additions, so of course zero ship — it is true *by construction*. Say this before the examiner does. The informative number is the other one: **gate-OFF = 40**, i.e. how often a carefully grounded prompt alone would have shipped a falsehood. The contribution is the measurement, not the tautology.

2 · Problem framing

why the obvious version of this project is the wrong one

The naïve framing is “use an LLM to improve a résumé’s match score”. That framing is unfalsifiable and unsafe, for a precise reason: the objective and the failure mode point in the same direction. If the résumé lacks *Kubernetes* and the job wants *Kubernetes*, the single most effective way to raise the score is to add *Kubernetes* — which is also the exact

behaviour that makes the tool dangerous. Any evaluation that only measures match improvement will rank a fabricating system above an honest one.

So the project splits the system into two components with opposed incentives, judged separately:

- **A generator** — an LLM, temperature 0.6, deliberately creative, permitted to be wrong.
- **A verifier** — deterministic, LLM-free, permitted to be strict. Same inputs always produce the same verdict.

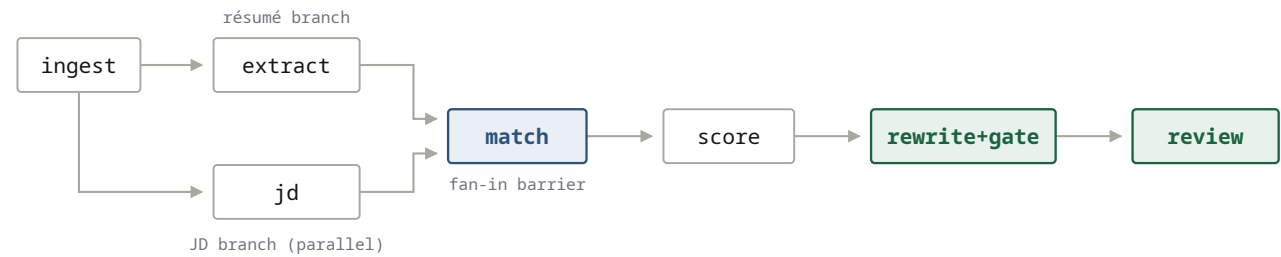
The generator proposes; the verifier disposes. This separation is what makes `fabrication_rate` a reproducible metric rather than a model's opinion of its own honesty. An LLM asked “did you fabricate?” is not evidence.

THE RESEARCH QUESTION, STATED EXACTLY

Not “can an LLM improve a résumé” (trivially yes), but: *how much improvement survives when every factual claim must trace back to the source document?* Answering that requires building the constraint first — which is the bulk of the engineering.

3 · Architecture

seven nodes · two parallel branches · one barrier



Note the JD box is drawn from `ingest` in the source graph, but JD analysis consumes the raw job-description text, not the résumé — the two branches are genuinely independent and overlap in wall-clock time.

WHY MATCH IS DECLARED WITH `DEFER=TRUE`

LangGraph's default triggering is *any-of*: a node fires as soon as any inbound edge lands. The JD branch is much shorter than résumé extraction, so the default would fire `match` at superstep 2 on a state containing `reqs` but no `resume` — an immediate crash — and then fire it *again* after extraction. `defer=True` makes it a true fan-in barrier. This is a real bug that was hit and fixed, and it is a good concrete answer to “what was hard?”

Table 3.1 – stage inventory. Note the alternation: every stochastic stage is followed by a deterministic one that checks it.

NODE	MECHANISM	OUTPUT	DETERMINISTIC
<code>ingest</code>	Docling layout-aware conversion	markdown + <code>ProvenanceMap</code>	yes
<code>extract</code>	LLM under <code>responseSchema</code>	<code>StructuredResume</code> + <code>*_prov</code>	no
<code>jd</code>	LLM under <code>responseSchema</code>	<code>RequirementSet</code> (must/nice)	no
<code>match</code>	mpnet cosine + keyword/fuzzy coverage	5-D <code>ComponentVector</code> , gaps	yes
<code>score</code>	Ridge regression	calibrated 0–100	yes
<code>rewrite</code>	LLM temp 0.6, then the gate	<code>TailoredResume</code> + report	no → gated
<code>review</code>	independent invariant re-check	<code>invariant_ok</code> , violations	yes

SERVING PATH (WHAT THE DEMO ACTUALLY RUNS)

The editor UI does *not* re-run `ingest/extract` — the user has already parsed and hand-edited the résumé. `run_optimize` in `src/rho/api/entry.py` composes the remaining stages directly and rebuilds provenance from the edited résumé’s own values via the real ingest path. **Consequence you must be ready to state:** in the editor flow, “sourced” means *traces to the current résumé*, not *traces to the original upload*. This is documented in the code, not hidden.

Jobs run on daemon threads in an in-process store (`src/rho/api/jobs.py`). A failed job is recorded as `state="error"` with the exception message — never swallowed — so a dead model cannot look like a clean run.

4 • The provenance chain (C1)

the substrate every guarantee rests on

Docling converts the PDF and reports geometry for every text item. Each non-empty item becomes a `SourceSpan` carrying document id, character offsets into the exported markdown, page number, bounding box, and the raw text. Together these form the `ProvenanceMap`.

```
# src/rho/ingestion/docling_adapter.py
# search_from advances so repeated strings map to SUCCESSIVE occurrences
# rather than all collapsing onto the first – a subtle correctness point
idx = md.find(content, search_from)
search_from = idx + len(content)
pm.add(SourceSpan(doc_id=doc_id, char_start=idx, char_end=idx + len(content),
                  page=page, bbox=bbox, raw_text=content))
```

RESOLUTION IS FUZZY, AND THAT IS DELIBERATE

```
# src/rho/extraction/provenance_attach.py
def find_prov(value, prov, threshold=90):
    # substring OR partial_ratio >= 90
    if v in raw or fuzz.partial_ratio(v, raw) >= threshold: hits.append(pid)
```

Exact matching would fail on ordinary formatting noise — a bullet glyph, a line break mid-phrase, a non-breaking space — and would reject truthful content. The cost of fuzziness is handled downstream: `find_prov` locating a span is treated as *necessary but not sufficient* by the gate (§6).

MEASURED, NOT ASSUMED

Provenance-attachment accuracy — does the attached `prov_id` point at the *correct* span, not merely at some span — is **0.874** on the public human-annotated set and **0.894** on the synthetic set. Every downstream guarantee inherits this ceiling. Roughly one span in eight is imperfect, which means a truthful claim whose span was mis-attached can be wrongly rejected by the gate.

5 • Extraction and JD analysis

the two stochastic stages, and how each is constrained

5.1 Structured extraction

Gemini's `responseSchema` provides genuine constrained decoding — the model cannot emit a field the schema forbids, nor a `priority` outside `{must, nice}`. Pydantic validation still runs behind it, because **a schema constrains shape, never truth**. That sentence is the whole point and is worth memorising.

Three reliability tiers are commonly distinguished, and you should be able to name them: prompt-only “return JSON” (unreliable), function-calling / JSON mode (syntactically valid, not schema-guaranteed), and constrained decoding (invalid tokens masked during generation). This system uses the third tier on every backend — Ollama's `format` parameter locally, `responseSchema` on Gemini.

5.2 JD requirement extraction

The JD prompt enforces requirement *granularity*, which turns out to matter enormously:

- Each ``text`` MUST be a short skill/tool token of 1-4 words, as it would appear in a résumé's skills list. Name the skill; do not restate the sentence.
GOOD: "Python", "AWS", "customer service", "SQL"
BAD: "must be authorized to work in the usa"
- Skip boilerplate that is not a skill: work authorisation, location, age.
- ``priority``: "must" for required items, "nice" for preferred/bonus.
If the text does not mark it optional, use "must".

Why the length cap exists. Coverage matches requirement text literally against résumé strings. A sentence-length requirement can never match anything, which would silently pin `keyword_coverage` and `fuzzy_coverage` at 0 for every résumé — a failure that looks like a scoring bug but is actually a JD-parsing bug. Cap is `_MAX_REQUIREMENT_WORDS = 5`.

Why unmarked defaults to “must”. Conservative: over-counting must-haves depresses the score rather than inflating it. Given the whole thesis is “do not flatter the candidate”, the default errs against the user, not for them.

6 · The fabrication gate (C3)

the core contribution – four layers, only the last two trusted

Layer 1 — schema constraint

Prevents malformed output. Prevents no falsehood whatsoever. Included for completeness only.

Layer 2 — prompt grounding

```
# src/rho/rewrite/gemini.py
- The MASTER RÉSUMÉ below is the ONLY source of truth.
- You MAY add a skill ONLY when the master résumé already demonstrates it
  in a work bullet, a project bullet, or a project's tech list.
- You MUST NOT invent skills, tools, employers, titles, metrics, dates.
- Return EVERY work entry and EVERY project, and KEEP ALL bullets.
- If a target requirement cannot be satisfied truthfully, leave it
  unsatisfied. An honest gap is the correct output;
  a fabricated match is a failure.
```

THE PROMPT IS NOT A SAFETY MECHANISM – IT IS A REQUEST

With this exact grounding in place and the gate disabled, **40 unsourced claims still shipped** across 30 pairs on Gemini, and **31** on qwen2.5:14b. That measurement is the entire justification for layers 3 and 4. If an examiner asks “why not just write a better prompt?”, the answer is: *that was the experiment, and it is the condition that fails.*

Layer 3 — the deterministic gate

Applied to every field after the rewriter returns, with no further LLM call. Given the same (tailored, source, prov) it always renders the same verdict. The gate distinguishes two field classes because they fail differently.

SHIPS	REJECTED AND COUNTED
- Value already in source — a reorder, not an edit. Not counted as an edit at all. - Added value where one span supports every content word (<code>partial_ratio ≥ 88</code>) - Bullet with <code>token_set_ratio ≥ 60</code> to a source bullet and no new hard tokens - Inflections — <code>optimising</code> / <code>optimised</code> collide under a crude stemmer	- Skill, tool, or certification with no supporting span - Bullet introducing an unsourced capitalised term, acronym, or digit run - Work entry whose company or title fails — entire entry dropped - Project whose name fails — dropped, never shipped as a real-looking shell

WHY BULLETS NEED A DIFFERENT TEST FROM SCALAR FIELDS

`find_prov` scores with `partial_ratio`, which finds the best-matching *substring*. So a short span ("Engineer") scores 100 against any long bullet that happens to contain that word — a wholly fabricated bullet would pass on one incidental match. Bullets are therefore compared whole-string with `token_set_ratio` against source bullets: rephrasing survives, new claims do not.

WHY SIMILARITY ALONE IS INSUFFICIENT

A long invention that recycles the source's vocabulary scores well — the worked counter-example in the code is “*Led a team of 40 Teradata engineers*”. So a bullet must *also* introduce no unsourced hard-content token. Both conditions are required:

```
# src/rho/rewrite/verifier.py
nearest = max(fuzz.token_set_ratio(b, s) for s in self._src_bullets)
if nearest >= 60 and not self._unsourced_hard_tokens(bullet):
    return True # a rephrasing of something already claimed
self.rejected.append(RejectedEdit(added_text=bullet, reason="no supporting prov_id"))
```

WHAT COUNTS AS “HARD CONTENT”

Skills, tools, employers, titles, certifications, dates, institutions, project names, numbers. *Not* prose — the rewriter is *supposed* to rephrase summaries and bullet phrasing. The regex is `[A-Z][A-Za-z0-9+#. ]+ | [A-Z]{2,} | \d[\d, . $%/- ]*`: capitalised terms mid-sentence, all-caps acronyms, digit runs. The leading word is skipped because sentence case capitalises whatever verb opens the bullet.

Table 6.1 – every threshold, its value, and its derivation. Expect to be asked “why that number?” for at least one of these.

CONSTANT	VALUE	APPLIES TO	WHY THIS VALUE
<code>_BULLET_SIMILARITY</code>	60	whole bullets	Measured on corpus rewrites: genuine rephrasings score 68–87 , inventions 37–42 . 90 rejected every real rephrasing. 60 sits in the empty band between the two classes.
<code>_WORD_MATCH</code>	88	added scalar values	Every word of an added value must appear in the supporting span. Guards the “ Engineer supports Staff Engineer ” promotion attack.
<code>find_prov_threshold</code>	90	span resolution	Tolerant enough for formatting noise; deliberately treated as necessary-not-sufficient by the gate.
<code>sem_hi / sem_lo</code>	0.65 / 0.45	requirement status	Provisional and untuned — never swept against a labelled match set. Declare this as a limitation before you are asked.

STEMMING – WHY A CRUDE ONE IS CORRECT HERE

A hand-rolled suffix stripper, not a real stemmer, and not a new dependency. The gate only needs `optimising/optimised` to collide, not linguistic correctness. Requiring every *word* to be sourced would reject truthful rewording (`improve` → `enhance`); requiring none would let inventions through. Stemming hard tokens only is the compromise.

Layer 4 — independent re-check

The `review` node re-verifies the *shipped* résumé from scratch: every hard-content token must resolve to at least one `prov_id`. This deliberately duplicates the gate. If it ever fires, the gate leaked. It reports `invariant_ok=False` with the offending values rather than raising, so a leak is surfaced and logged, never silently swallowed.

IT HAS ACTUALLY CAUGHT SOMETHING – USE THIS EXAMPLE

On a real pipeline run, extraction turned a plain year “2020” into a full ISO timestamp “2020-01-01T00:00:00Z”, which fell just under the fuzzy threshold and so resolved to no span. The reviewer flagged `invariant_ok=False` rather than shipping it. On the same run the gate independently rejected a genuinely fabricated skill, “Data Engineering”, proposed with no supporting span. Both mechanisms doing their job on an unstaged generation.

7 • Matching and calibrated scoring (C2)

predicting a real ATS rather than inventing a number

7.1 The feature vector

Five dimensions, in a frozen order (a saved calibrator is only valid for this ordering): `keyword_coverage`, `semantic_similarity`, `fuzzy_coverage`, `must_have_coverage`, `nice_have_coverage`.

`semantic_similarity` is the *mean best-match cosine* across requirements over `all-mpnet-base-v2` embeddings — deliberately not a *match rate*, so it stays a continuous signal distinct from the coverage fields. A requirement counts *present* at cosine ≥ 0.65 , *weak* at ≥ 0.45 .

Coverage searches the *whole résumé* — skills, certifications, achievements, headline, summary, work titles/companies/bullets, education — not just the skills list. Real requirements are often evidenced in a bullet, and matching against skills alone pins both coverage signals near zero.

7.2 Why a calibrator at all

A hand-weighted average of those five would be a number *we invented*. Instead a **Ridge regression** ($\alpha=1.0$, scikit-learn) is fit to predict the output of `ats-screener` — a rule-based, deterministic, six-profile ATS simulator, vendored at pinned commit `4105f77` and driven through a headless Node runner so results reproduce exactly. No LLM, no network.

TARGET SELECTION – THE MOST DEFENSIBLE DECISION IN THE PROJECT

`ats-screener` scores five dimensions, but only `keywordMatch` varies with the job description; formatting, sections, experience and education score the résumé *on its own*. On this corpus `keywordMatch` contributed only ~2–11 of ~45 composite points. Calibrating against the composite produced a target our match features correlated **negatively** with — we would have been predicting résumé-intrinsic quality. Restricting to the JD-dependent dimension flips every correlation positive.

Why Ridge and not something richer? 199 pairs, five features. A higher-capacity model overfits, and linear coefficients stay inspectable — an auditable pipeline should not terminate in an uninterpretable scorer. The point of C2 is not the model family; it is that the score is anchored to what a real ATS does.

No imputation. Documents are dropped, never filled, when no engine scores them, so the fit only sees real observations.

7.3 The display transform

The calibrated score sits in a narrow low band (held-out target mean 13.36, sd 4.88; practical range ≈ 7 –27), so a strong résumé scores ~21/100 and reads as a failing grade. `to_display_score` linearly maps $7 \rightarrow 0$ and $27 \rightarrow 100$ for the UI while the raw calibrated value is preserved for reproducibility. **It is a presentation transform, not a model** — it invents no signal, it stretches an axis. If the calibrator is refit, these anchors must be re-measured.

8 • Evaluation methodology

what was measured, on what data, and which numbers to trust

8.1 Three gold sets, deliberately not averaged

1a • synthetic (n=120)

Labels and provenance spans exact by construction. Templated text is cleaner than reality, so this is an **upper bound**, not an estimate.

1b · hand-labelled real
(n=25/30)

Labelled by the implementing author, **not independently verified**. A reality check, not evidence.
Name and company are unlabelable — the corpus anonymises them.

1c · public human-annotated
(n=143/150)

kenslang/resume-ner-labelled . Annotations independent of this system and its author. **This is the headline C1 number** and the only one to quote unqualified.

Metric families are kept separate on purpose. `field_f1` is set-based (order and duplication do not matter).

`long_text_f1` is token-overlap, because exact match is meaningless for generated prose. Averaging prose into the entity score would hide that long-text is the hardest field class. Both-empty scores 1.0 — the extractor correctly reporting an absent section is a right answer, not a miss.

8.2 Calibration and fabrication corpora

Calibration: 199 usable résumé–JD pairs (139 train / 60 held-out), paired from public résumé and job-posting corpora, weighted **80% toward same-category pairings** so there is genuine overlap to find — random pairing would make the task trivially separable and the correlation meaningless.

Fabrication: 30 corpus-backed pairs. Crucially the adversarial pressure — *which* requirements are missing — is generated by the system's own JD-analysis and matching stages, **not a hand-written list**. Hand-picking the gaps would let the author choose the difficulty.

9 · Results in full

every table you may be asked to reproduce or defend

Table 9.1 – extraction quality (C1), gemini-3.1-flash-lite. Quote 1c; qualify 1a and 1b.

GOLD SET	N SCORED	SKILLS F1	TITLE F1	INSTITUTION F1	PROV. ACC.
1a synthetic (upper bound)	120/120	0.999	1.000	1.000	0.894
1b real corpus (unverified)	25/30	0.443	0.931	0.693	—
1c public human-annotated	143/150	0.753	0.935	0.917	0.874

1c also: certification F1 0.802; skills precision 0.802 / recall 0.749 (macro-averaged per document).

Table 9.2 – calibration (C2), held-out split, target = ATS simulator's JD-dependent dimension.

BACKEND	N USABLE	CALIBRATED MAE / P	COSINE MAE / P	WALL-CLOCK
gemini-3.1-flash-lite	199/199	3.25 / 0.333	27.58 / 0.229	836 s
gemma3:4b (Ollama)	199/200	3.17 / 0.328	26.00 / 0.168	39,162 s (10.9 h)

Read p, not MAE, as the headline. The cosine baseline emits 0–100 while the true target band is narrow and low, so most of its MAE is scale mismatch rather than ranking failure. The two backends are within run-to-run noise of each other (a repeat of the open-model run alone gave $p \in \{0.328, 0.440\}$ on different splits), so the small spread is *not* a backend effect — do not claim it is. The real finding here is throughput: **47× faster wall-clock** on the hosted backend.

Table 9.3 – fabrication gate (C3), 30 adversarial pairs. The cross-backend row is what makes this a structural claim rather than a quirk of one model.

BACKEND (REWRITER)	PAIRS	GATE OFF	GATE ON	MEAN FAB. RATE	PAIRS AFFECTED
gemini-3.1-flash-lite	30/30	40	0	0.248	10/30
qwen2.5:14b (Ollama)	30/30	31	0	0.407	14/30

Gemini fabricates *more per affected pair* but on *fewer pairs* — more aggressive where it fabricates at all, not more conservative overall. Rejected examples are substantive, not near-miss paraphrases: invented skills (*Microsoft Word*, *Microsoft Excel*, *Project Management*), invented soft-skill claims (*Team Leadership*), invented bullets (“*Performed tasks requiring basic math... using 10-key*”), and on the open model, invented employers, fabricated metrics, and a leaked prompt-reasoning artifact.

ABLATION – DOES THE PROVENANCE CHAIN EARN ITS COMPLEXITY?

Replace span-resolved provenance with the cheaper substitute a provenance-free system would use — a case-insensitive substring search for the value anywhere in the source text. It catches **36 of 40** rejections. **Four would ship:** additions that appear *somewhere* in the source but are not supported *where the rewriter used them*. Those four are the measured value of resolving to an actual span.

LATENCY AND COST

Extraction median **3.4 s** (n=288, min 1.8 s, max 19.4 s). Full pipeline end-to-end **15.7 s** hosted vs **268.5 s** open-model (17×). API spend **\$0.00** — a free-tier quota artifact, *not* an architectural property. Say this unprompted; self-hosted compute is free at any scale, the hosted tier is not.

10 • Failure modes and limitations

state these before the examiner finds them – it is worth marks

10.1 The deterministic extraction failure

12/300 documents (4%) failed extraction outright rather than scoring poorly, and are excluded from the tables rather than counted as zero. Root cause confirmed by direct reproduction and by two independent full runs producing *identical* failures on *identical* document ids — temperature is 0 for extraction, so this is deterministic, not sampling noise.

The model enters a **degenerate repetition loop** when employment dates are formatted ambiguously across lines (e.g. "03/2015\nto\n07/2017"), repeating a malformed ISO-timestamp fragment until it exhausts its output-token budget; the response truncates mid-string and is not valid JSON. Per-document try/except isolation degraded this to “12 skipped and logged” rather than a crashed run — but it is a real gap between attempted and successful throughput that a deployment must plan for. *Not observed on the open-model backend.*

10.2 Everything else

1. **The scorer is also the evaluator.** Improvement is measured against a scorer this project calibrated. $p = 0.333$ is a modest predictor, not a faithful ATS simulation. Movement against a real ATS or human raters would be far stronger evidence.
2. **The coverage function changed after the calibrator was fit.** Coverage is now position-weighted (§11). The shipped calibrator was fit against the *previous* unweighted function. **It must be refit and the ablations re-run before these figures go into a written report.** This is the most important caveat in this document.
3. **Guarantees inherit provenance accuracy (0.874).** A truthful claim whose span was mis-attached gets rejected — a false positive against the user, which is currently *unmeasured*.
4. **Omission is not gated.** The gate checks nothing is *invented*. It does not check nothing is silently *dropped*. Bullet preservation is a prompt instruction, not an enforced invariant; a truncated response yields a shorter résumé with no error raised.
5. **Semantic thresholds untuned.** 0.65 / 0.45 are original design defaults, never swept against a labelled match set. Provisional.
6. **Fabrication benchmark scale.** 30 pairs. Enough to demonstrate the effect and (holding across two backends) to argue it is structural; not enough for a confidence interval on the mean fabrication rate.
7. **Single language and domain.** The hard-token regex treats Latin capitalisation as a factual-content signal; it would not transfer to a caseless script.
8. **No fine-tuned small-model baseline.** A plausible alternative to prompting a general LLM; the vLLM/Outlines path exists in the codebase but was never exercised on a CUDA host.

11 • Recent engineering work

changes made after the paper's numbers were generated – know which is which

11.1 Deployment bugs found and fixed

`libxcb.so.1` missing

`rapidocr` pulls `opencv-python`, which bundles Qt GUI bindings linked against X11 — absent from a slim container. Fixed by swapping to `opencv-python-headless`. The uninstall must precede the install: both ship the same `cv2/` directory.

`InvalidCxxCompiler`

Docling's layout model runs under `torch.compile`; the inductor backend shells out to `g++` to JIT kernels. Fixed with `TORCHDYNAMO_DISABLE=1` — eager execution, same results, no ~200 MB

compiler layer.

MATCH SCORE 0/100

`.dockerignore` excluded `eval/`, so `eval/calibrator.joblib` was never in the image and `score_node` fell back to 0.0 for every résumé. Fixed by shipping the artifact and making the path CWD-independent.

Health check blind spot

Warm-up deliberately swallows exceptions so a bad warm-up cannot down the server — correct, but it meant the container reported *healthy* while every upload failed. Worth a readiness flag.

11.2 A design change with research consequences

Optimise could not move any component score. Cause: the rewriter's only permitted skills edit is *reordering*, and coverage flattened the résumé into an unordered *set* of words — so every before/after pair was identical by construction. Two changes were made:

- **Scorer:** coverage is now position-weighted (lead 1.0 / mid 0.8 / tail 0.55). Tail weight stays well above zero because burying a skill does not make it untrue.
- **Rewriter:** may now promote a skill into the skills list *only* if the résumé already demonstrates it in a bullet or tech list. The gate enforces this independently — the prompt is not trusted.

BE HONEST ABOUT THIS IF ASKED

The scorer was made responsive to the exact edit the rewriter makes. That is a defensible modelling choice — position genuinely reflects how a skills list is read — but it means part of any post-change improvement is measured against a scorer tuned after the fact. The calibrator must be refit before these numbers are reported. Volunteer this; do not let it be discovered.

11.3 Worked example (live run, DevOps Engineer Intern JD)

COMPONENT	BEFORE	AFTER
Match score (display)	30	31
Keyword match	30.0%	31.2%
Semantic match	60.0%	63.8%
Nice-to-have coverage	0.0%	50.0%
Must-have coverage	76.5%	76.5%

One edit survived: **Azure** promoted into the skills list. It was already evidenced in a work bullet (“...*tracking work items and sprint deliverables in Azure DevOps*”) but missing from the skills section — under-reported, not new. The gate verified it against that span. `total_edits 1`, `verified 1`, `rejected 0`. Must-have coverage did not move, correctly: the résumé genuinely lacks AWS, GCP, Terraform, Ansible, Jenkins, ELK.

HOW TO DEFEND A +1 POINT IMPROVEMENT

This is not a weak result being dressed up — **it is the finding**. Truthful tailoring has a low ceiling, and that ceiling is set by what the candidate has actually done. A system that moved this résumé from 30 to 85 would be lying. The contribution is a pipeline that makes the ceiling visible and measurable instead of papering over it with fabrication.

12 · Anticipated questions

rehearse these — most are hostile-but-fair

Q1 Isn't zero fabrication trivially achieved by not rewriting at all?

Yes, and that is exactly why gate-ON = 0 is not presented as the result. The pair of numbers matters: the gate removed 40 fabrications *while* the rewriter still produced verified edits. The right follow-up experiment — which I have not run — is **retained gain**: what fraction of the ungated score improvement survives gating. I would need that number before claiming the gate is cost-free.

Q2 Why not just use a better prompt?

That is the gate-OFF condition, and it is the one that fails. The prompt already says “an honest gap is the correct output; a fabricated match is a failure”, and 40 unsourced claims shipped anyway on Gemini, 31 on qwen2.5. Prompts express preferences; they do not enforce constraints.

Q3 Your thresholds — 60, 88, 90 — look arbitrary.

They are empirical and their derivations are in the source. The bullet threshold sits in a measured gap: genuine rephrasings score 68–87, inventions 37–42, and 90 rejected every real rephrasing. They are tuned on this corpus, though, and would need re-derivation on another — I would not claim they transfer.

Q4 Why Ridge regression rather than a neural scorer?

199 pairs and five features — a higher-capacity model overfits. And linear coefficients stay inspectable, which matters for a pipeline whose entire selling point is auditability. Terminating an auditable chain in an uninterpretable model would be self-defeating.

Q5 Is the improvement statistically significant?

Not established, and I would not claim it. The worked example is one résumé, one JD, +1 display point. The corpus-level distribution of the effect has not been computed. What is well-supported is the C3 result, which held exactly across all 30 pairs on two independent backends.

Q6 What stops the *extraction* LLM fabricating, before the gate ever runs?

Partly the schema, partly `attach_provenance` — an extracted value resolving to no span carries an empty `prov` list and cannot support a later edit. But extraction is *measured, not gated*: skills F1 0.753 on real data. A hallucinated extraction that happens to resolve to a span would propagate. **This is the weakest link in the chain** and I would name it as such rather than defend it.

Q7 Why is skills F1 only 0.753 when the other fields are above 0.9?

Three causes, and I measured their proportions. The metric is *exact* set-match, so granularity disagreements count as both a false positive and a false negative — of 881 misses, **27% are substring relations** (gold “*digital marketing strategies*” vs predicted “*marketing*”). Relaxing to partial credit lifts F1 to **0.805**, so ~0.05 is purely definitional. Second, the extractor over-segments compounds (“*microsoft office suite*” → excel, word, outlook). Third, this corpus is nursing/HR/sales, and gold includes soft skills like “*detail-oriented*” that are arguably not skills. I would report both exact and partial-credit F1 rather than silently switching metric.

Q8 Why should I believe the gate generalises beyond the models you tested?

Because it never inspects the model. It compares proposed output against source spans using fuzzy string matching and a hard-token regex — no learned component, no model-specific assumption. The cross-backend result (Gemini and qwen2.5:14b, both gate-ON = 0) is evidence, not proof. A stronger test would be an adversarial rewriter explicitly instructed to evade the gate; I have not run that.

Q9 What happens if Docling misreads the PDF?

Provenance is built from Docling's output, so a mis-read propagates — measured attachment accuracy is 0.874, not 1.0. The failure is *conservative*: a missing span causes truthful content to be rejected, not fabricated content to be accepted. The gate errs against the user, which is the correct direction for this application.

Q10 Is this actually useful, given +1 point?

As a product, marginally. As a result, yes — it quantifies the ceiling on honest tailoring, which is a number nobody in this space reports because the incentive is to fabricate past it. The system's real utility is diagnostic: it tells a candidate *which* requirements they genuinely fail (must-have coverage 76.5%, six missing tools), which is actionable career information rather than a cosmetic edit.

Q11 Why LangGraph rather than a plain function pipeline?

Honestly, for a linear chain it is not required. It earns its place at the fan-in: résumé parsing and JD analysis are independent, and the barrier semantics (`defer=True`) plus per-node checkpointing are things I would otherwise hand-roll. For a seven-node graph it is a reasonable but not essential choice, and I would not oversell it.

Q12 What would you do differently?

Measure the gate's false-positive rate from the start — truthful claims wrongly rejected is the gate's real cost to a user and it is still unmeasured. And I would have frozen the scoring function before fitting the calibrator, so that late changes to coverage did not invalidate the fit.

13 · Code map

where to point if asked to show something

<code>src/rho/graph/__init__.py</code>	LangGraph topology, <code>defer=True</code> fan-in barrier
<code>src/rho/graph/nodes.py</code>	node functions; calibrator loading in <code>score_node</code>
<code>src/rho/ingestion/docling_adapter.py</code>	<code>PDF</code> , <code>markdown</code> + <code>ProvenanceMap</code> , span offsets
<code>src/rho/extraction/provenance_adapter.py</code>	<code>find_prov</code> , <code>attach_provenance</code>
<code>src/rho/rewrite/verifier.py</code>	the gate — <code>_Ledger</code> , thresholds, <code>verify_against_source</code>
<code>src/rho/rewrite/tokens.py</code>	<code>hard_content_tokens</code> — what is checkable
<code>src/rho/rewrite/gemini.py</code>	rewrite prompt and response schema
<code>src/rho/jd/gemini.py</code>	JD requirement extraction, granularity rules
<code>src/rho/matching/__init__.py</code>	<code>match</code> , component vector, gap statuses
<code>src/rho/matching/coverage.py</code>	keyword/fuzzy coverage, position weighting
<code>src/rho/ats/calibrator.py</code>	Ridge calibrator, frozen feature order
<code>src/rho/ats/aggregate.py</code>	target definition — why <code>keywordMatch</code> only
<code>src/rho/ats/display.py</code>	the 7–27 → 0–100 presentation transform
<code>src/rho/graph/review.py</code>	independent invariant re-check
<code>src/rho/api/entry.py</code>	<code>run_optimize</code> ; editor-path provenance rebuild
<code>eval/run_all.py</code>	reproduces every table; F1 and provenance metrics

eval/metrics.py

eval/RESULTS.md

generated results of record

REPRODUCING THE NUMBERS

`python -m eval.run_all` regenerates every table from the datasets in `eval/datasets/`; extractions are cached per document under `eval/cache/extraction/`, keyed by `sha1(backend \0 text)`, so a re-run reuses them. Test suite: **233 passing, 4 skipped**.

Sources – `eval/RESULTS.md` (generated by `python -m eval.run_all, backend gemini`) · `docs/paper/paper.tex` · `src/rho/**` · `ats-screener` pinned at 4105f77 · worked example and error analysis from live runs against the local Docker stack.

Caveat of record: the position-weighted coverage change post-dates the fitted calibrator. Refit before publishing any score-delta figure.